Pathfinding Using Search Algorithms

Vida Sadati

In this presentation, we analyze classical search algorithms including BFS, DFS, UCS, and A*.

Using Python and an interactive graphical interface, these algorithms are applied to a city map of Kerman to visualize pathfinding processes while considering distance and traffic-based costs.

Steps of the Project

01

Nodes are placed on the city map to represent important locations and intersections.
These nodes define the vertices of the graph used for pathfinding.

Steps of the Project

01

Nodes are placed on the city map to represent important locations and intersections.
These nodes define the vertices of the graph used for pathfinding.

02

Paths are drawn between the placed nodes to represent possible roads.
Each path defines an edge connecting two nodes in the graph.

Steps of the Project

01

Nodes are placed on the city map to represent important locations and intersections.
These nodes define the vertices of the graph used for pathfinding.

02

Paths are drawn between the placed nodes to represent possible roads.
Each path defines an edge connecting two nodes in the graph.

03

Traffic values are assigned to each path to simulate real-world conditions. These values affect the traversal cost and influence the algorithm’s decisions.

Steps of the Project

01

Nodes are placed on the city map to represent important locations and intersections.
These nodes define the vertices of the graph used for pathfinding.

02

Paths are drawn between the placed nodes to represent possible roads.
Each path defines an edge connecting two nodes in the graph.

03

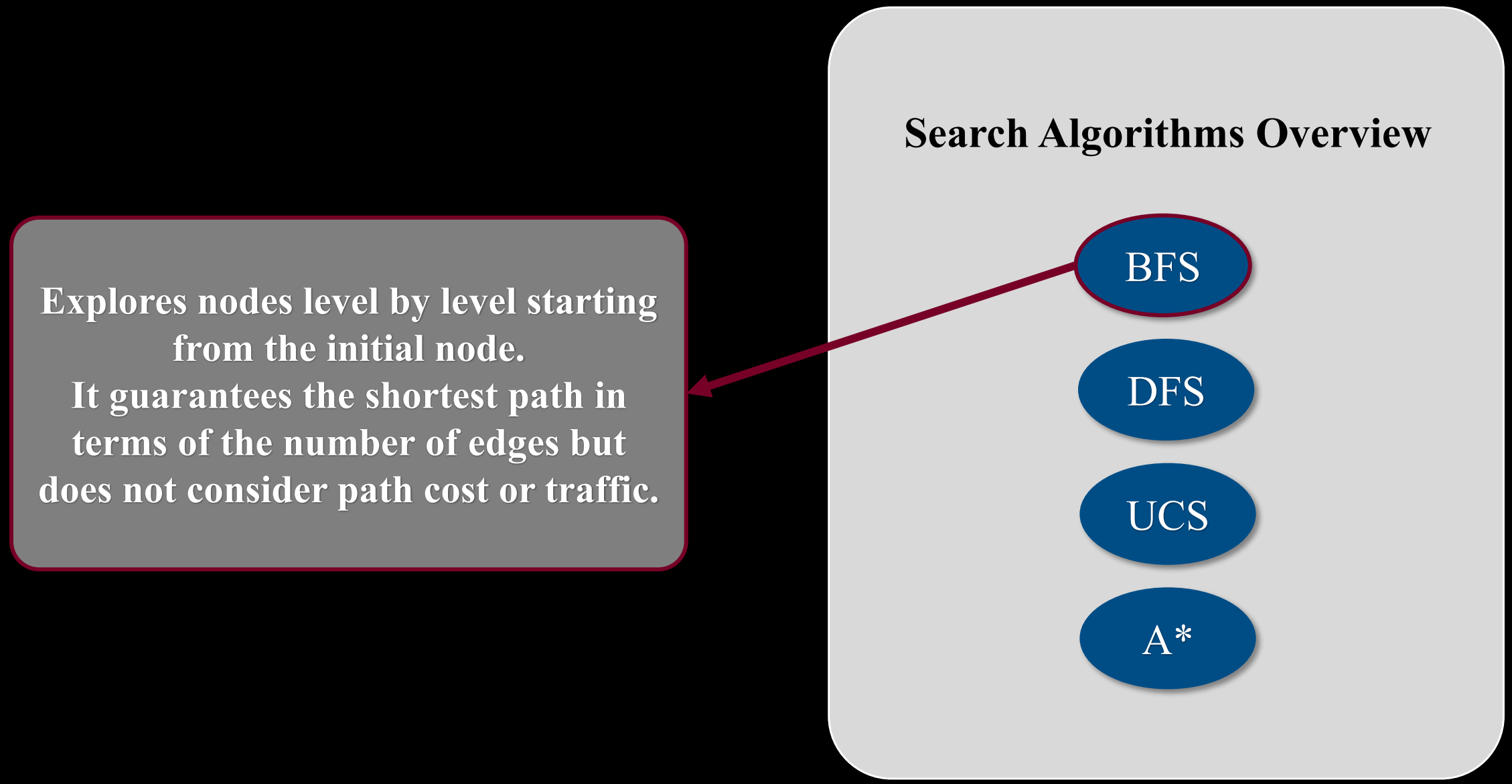
Traffic values are assigned to each path to simulate real-world conditions. These values affect the traversal cost and influence the algorithm’s decisions.

04

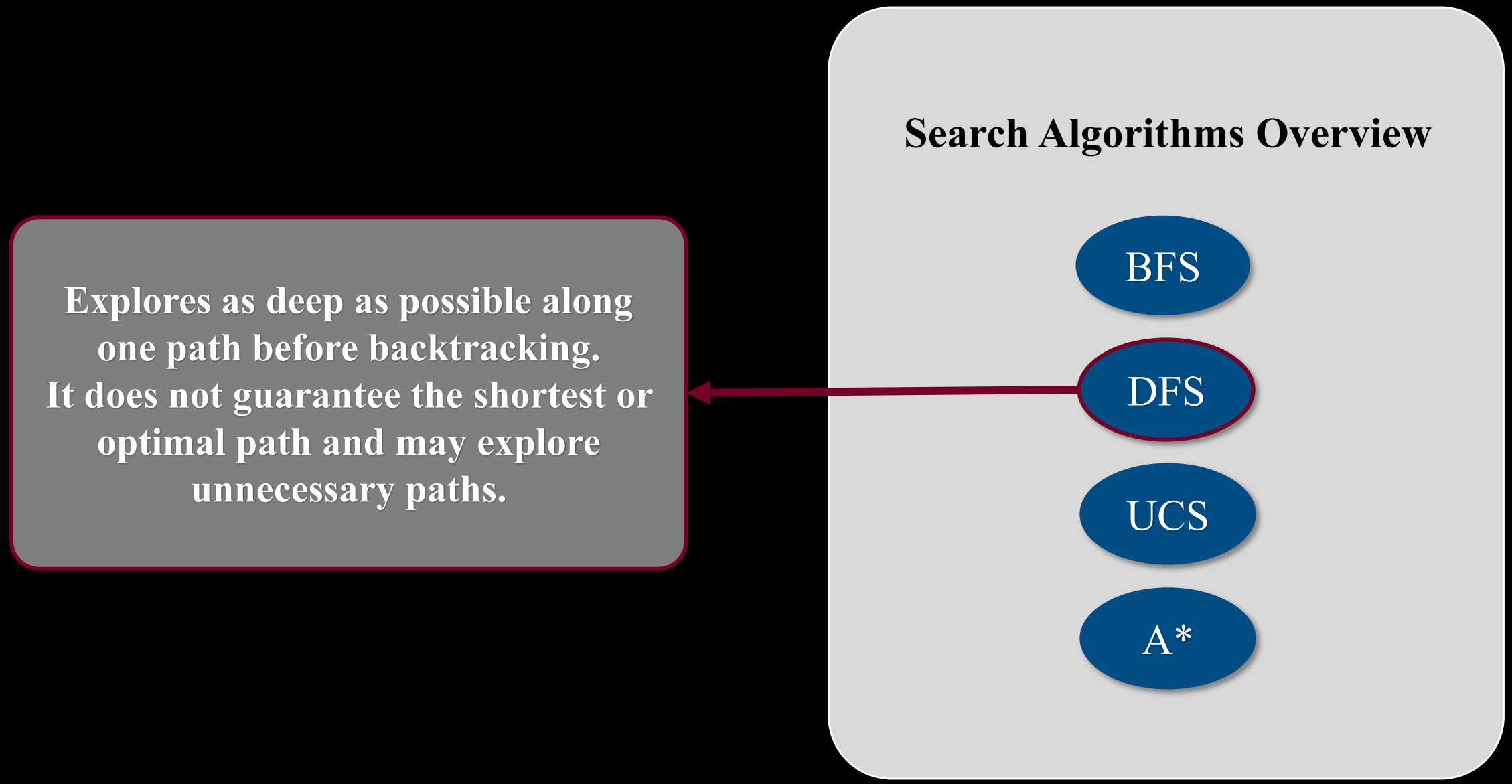
Search algorithms such as BFS, DFS, UCS, and A* are executed on the constructed graph. The traversal process and the final path are visualized directly on the city map.

Search Algorithms Overview

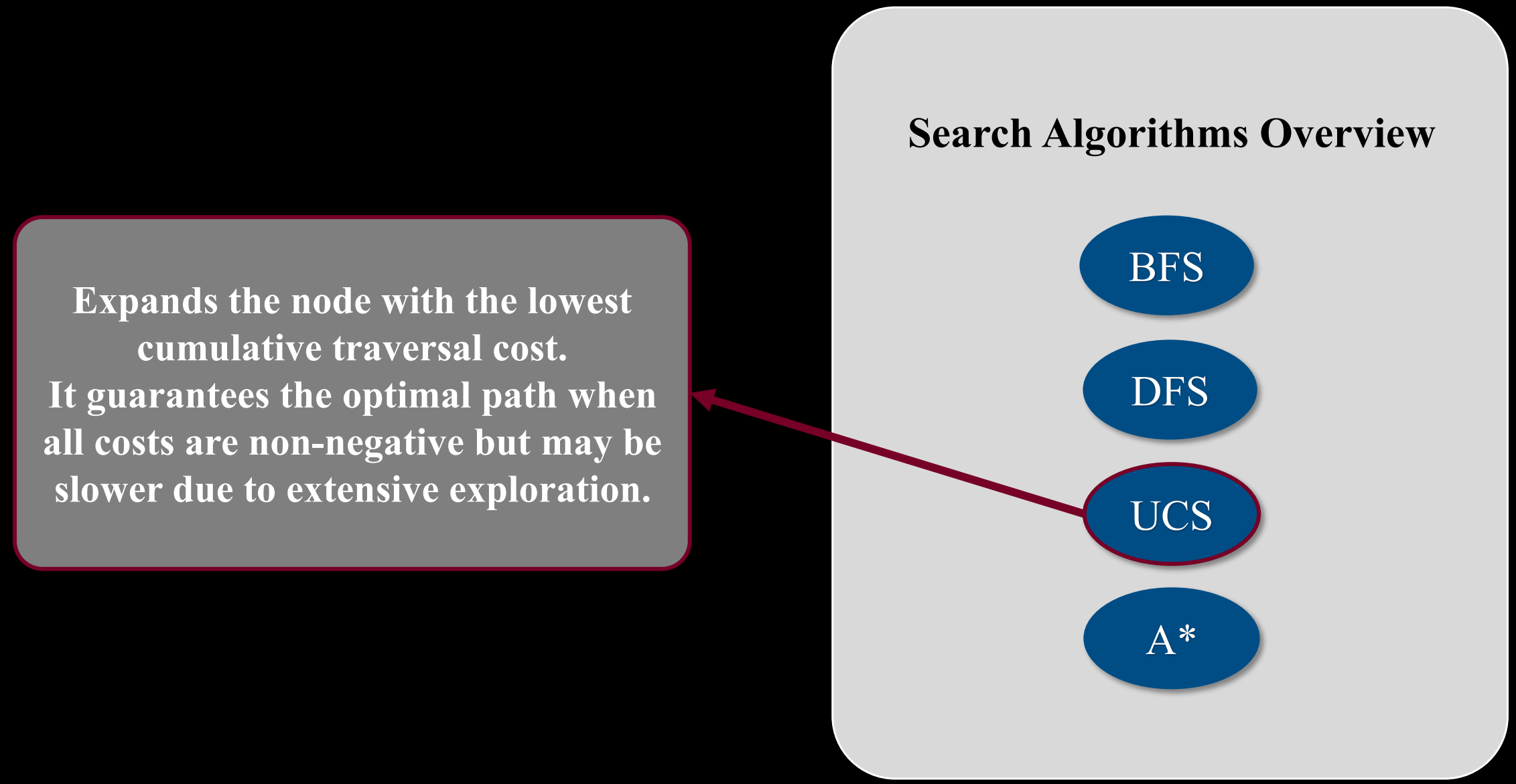
Search Algorithms Overview



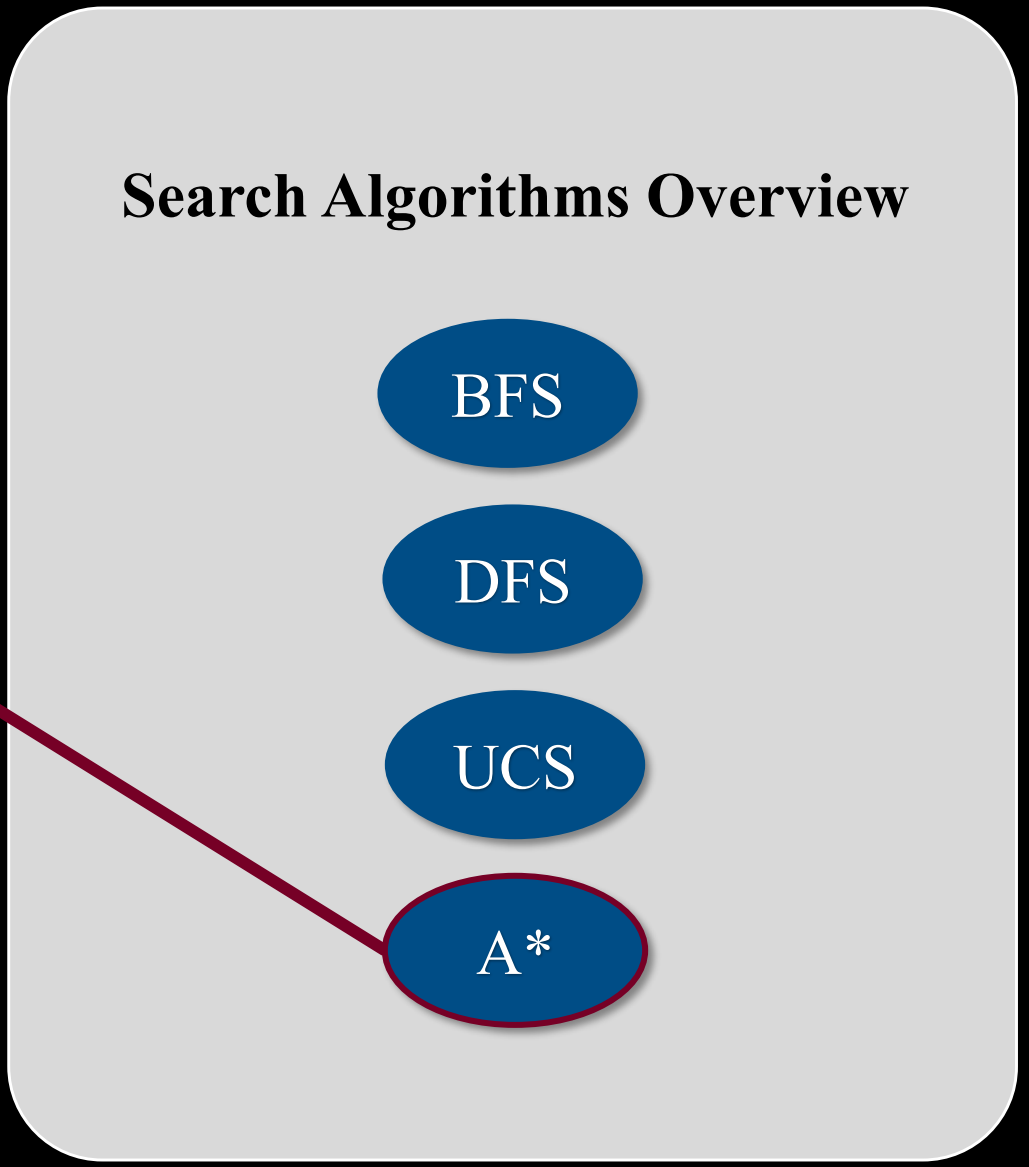
Search Algorithms Overview



Search Algorithms Overview

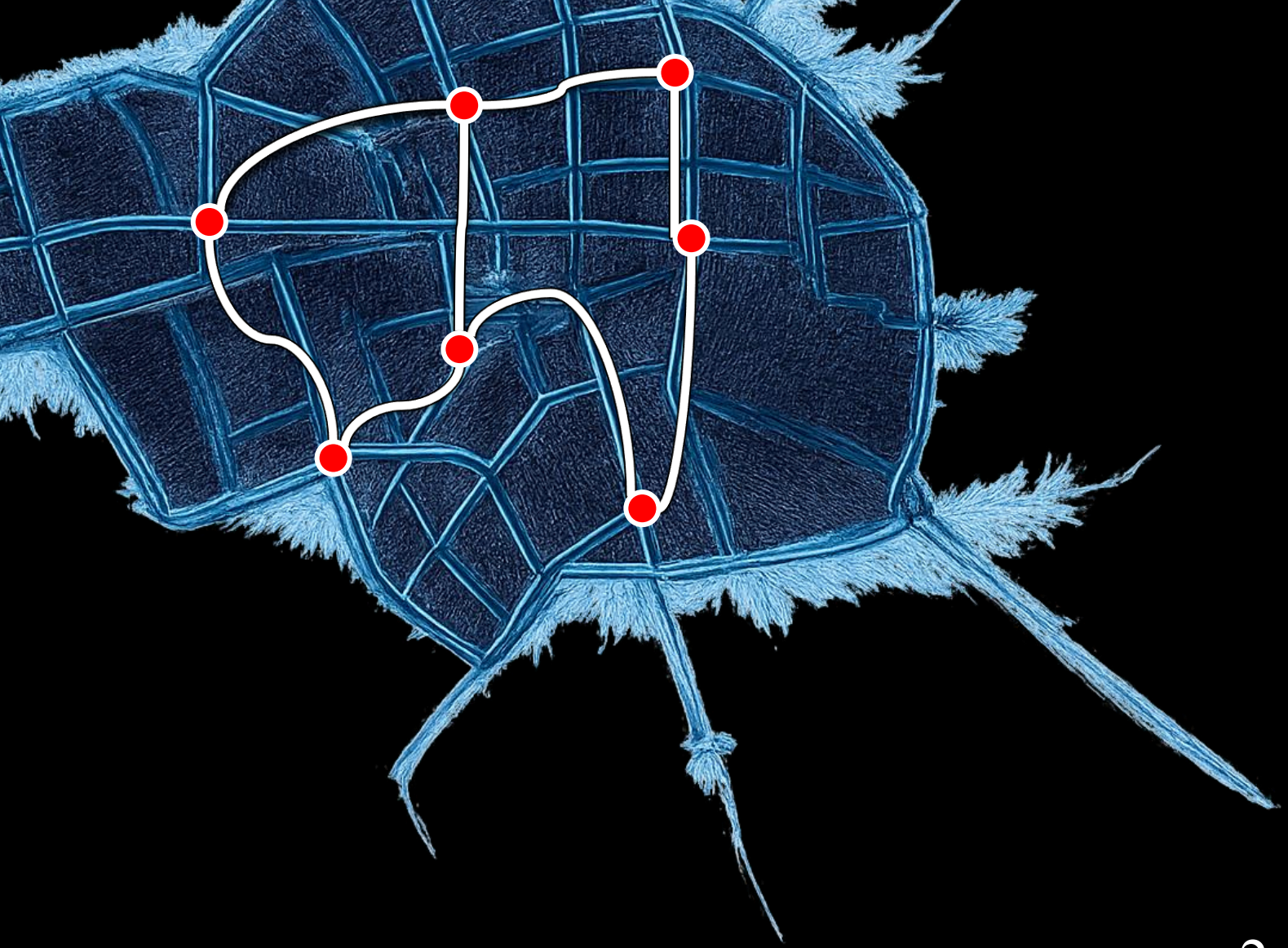


Uses the sum of the path cost so far and a heuristic estimate to the goal. This approach significantly reduces the search space while still guaranteeing an optimal path when the heuristic is admissible.



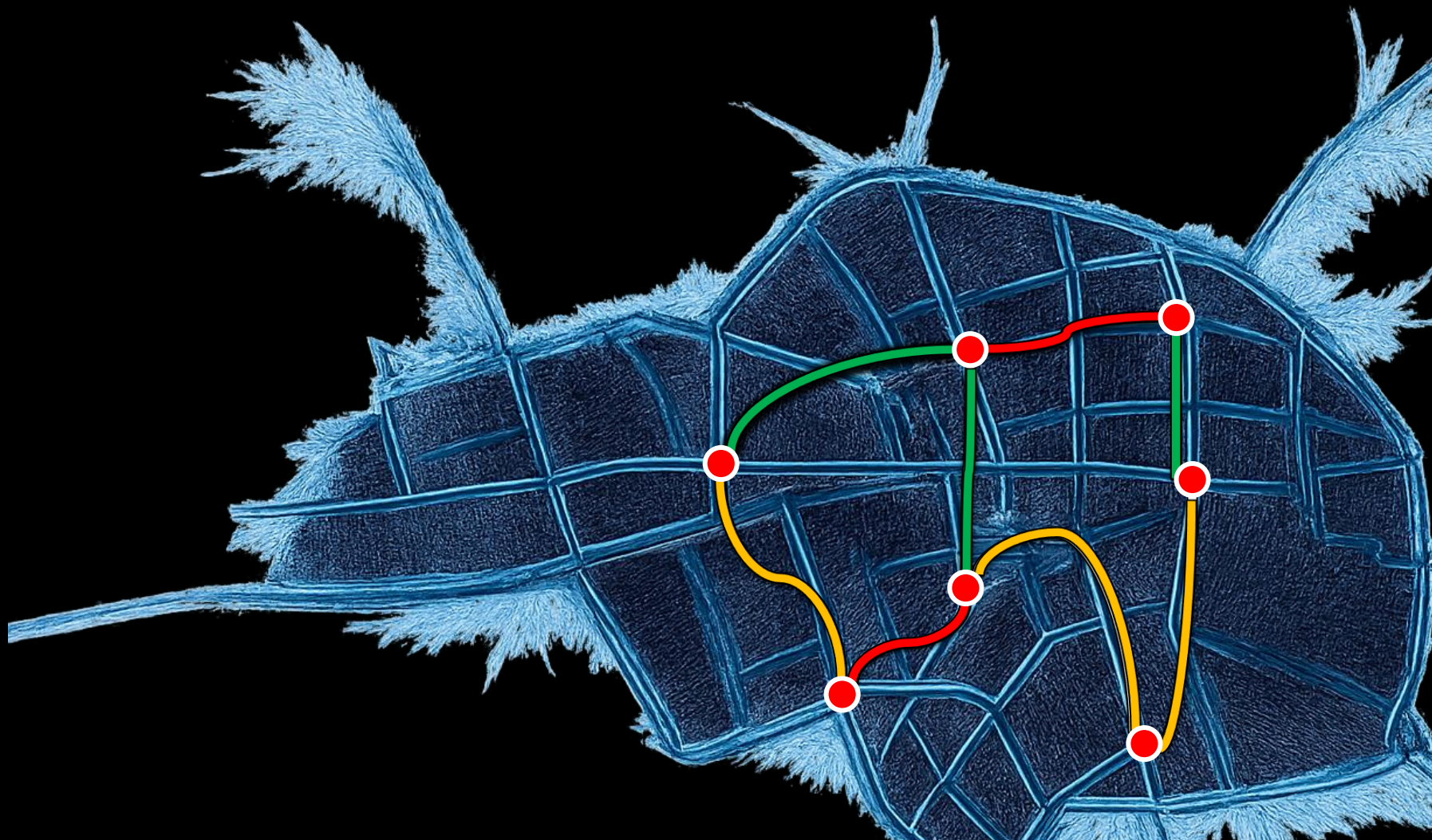
Build Map

1. Selecting nodes on the map



2. Drawing Paths Between Nodes

3. Traffic Value Assignment



Selecting nodes on the map

Selecting Nodes On The Map

Nodes can be placed using right-click. You can zoom using the mouse scroll wheel and drag the map by pressing D and using left-click.

Selecting Nodes On The Map Code

```python
# -------- Node mode --------
if self.mode == "node" and event.button() == Qt.LeftButton:
    super().mousePressEvent(event)
    return

pos = self.mapToScene(event.pos())

if self.mode == "node" and event.button() == Qt.RightButton:
    if len(self.nodes) >= MAX_NODES:
        print(f"❌ MAX NODES reached ({MAX_NODES})")
        return
    idx = len(self.nodes)

    circle = QGraphicsEllipseItem(
    pos.x() - NODE_RADIUS,
    pos.y() - NODE_RADIUS,
    NODE_RADIUS * 2,
    NODE_RADIUS * 2
    )
```

```python
    )

    circle.setBrush(COLOR_NODE_NORMAL)
    pen = QPen(Qt.white)
    pen.setWidth(1)
    circle.setPen(pen)
    circle.setZValue(11)
    self.scene.addItem(circle)


    label = QGraphicsTextItem(str(idx))
    label.setDefaultTextColor(Qt.yellow)
    label.setPos(pos.x() + 4, pos.y() + 4)
    label.setZValue(10)
    self.scene.addItem(label)

    self.nodes[idx] = {
        "pos": pos,
        "item": circle,
        "label": label
    }
```

Drawing Paths Between Nodes

Press P to enter path-drawing mode. Click on a node, drag the mouse to another node, and click again on the second node to complete the path.

Drawing Paths Between Nodes Code

```python
# -------- Path mode --------
elif self.mode == "path" and event.button() == Qt.LeftButton:
    node = self.find_node(pos, NODE_DETECT)

    if self.start_node is None:
        if node is None:
            print("❌ click on start node")
            return
        self.start_node = node
        self.drawing = True
        self.path_points = [self.nodes[node]["pos"]]
        self.path = QPainterPath(self.nodes[node]["pos"])

        outline_pen = QPen(QColor(70, 70, 70, 195), 12)
        outline_pen.setCapStyle(Qt.RoundCap)
        outline_pen.setJoinStyle(Qt.RoundJoin)
        self.path_outline = self.scene.addPath(self.path, outline_pen)
        self.path_outline.setZValue(4)

        # main path
        main_pen = QPen(QColor(255, 255, 255, 220), 6)
        main_pen.setCapStyle(Qt.RoundCap)
        main_pen.setJoinStyle(Qt.RoundJoin)
        self.path_item = self.scene.addPath(self.path, main_pen)

        self.path_item.setZValue(5)
        print(f"Start node: {node}")

    else:
        if node is not None and node != self.start_node:
            # snap to end node
            end_pos = self.nodes[node]["pos"]
            self.path.lineTo(end_pos)
            self.path_points.append(end_pos)

            length = self.path_length(self.path_points)
            traffic = 1.0  # Default traffic if not set yet

            traffic = self.get_traffic()

            edge = {
                "from": self.start_node,
                "to": node,
                "points": [(p.x(), p.y()) for p in self.path_points],
                "length": length,
                "traffic": traffic,
                "cost": length * traffic,
                "item": self.path_item,          # Main path
                "outline": self.path_outline     # Outline of the path
            }

            self.edges.append(edge)
            print(f"Path saved: {self.start_node} -> {node} | "
                  f"length = {length:.1f}, traffic = {traffic}, cost = {length * traffic:.1f}")

        # reset state
        self.start_node = None
        self.drawing = False
        self.path = None
        self.path_item = None
        self.path_points = []

super().mousePressEvent(event)
```

Traffic Value Assignment

After drawing a path, the user is prompted to assign a traffic value between 1 and 10.

1–3.5: Low,

3.5–6.5: Medium,

6.5–10: Heavy traffic.

Finally, press Q to save and finalize the input.

Traffic Value Assignment Code

```python
# ========================================================
# Traffic
# ========================================================
def get_traffic(self):
    # Request traffic value from the user
    traffic, ok = QInputDialog.getDouble(self, "Traffic", "Enter traffic value (0-10):", 1.0, 0, 10, 1)
    if ok:
        return traffic
    else:
        return 1.0  # Default traffic value

def start_traffic_input_mode(self):
    # Enable traffic input mode
    self.traffic_mode = True
    print("Traffic input mode activated. Please draw a path first and then enter traffic.")

def get_traffic_color(self, traffic):
    # CHANGE COLOE OF PATH BASED ON TRAFFIC
    if traffic <= 3.5:
        return COLOR_PATH_TRAFFIC_GREEN
    elif traffic <= 6.5:
        return COLOR_PATH_TRAFFIC_ORANGE
    else:
        return COLOR_PATH_TRAFFIC_RED
```

Implementation of Search Algorithms

Implementation of BFS

```python
# ============================================================
# Breadth-First Search (BFS)
# ============================================================
def bfs(self, start, goal):
    q = deque([(start, [start])])
    vis = set()
    self.steps = []

    graph = self.build_graph()

    while q:
        n, path = q.popleft()

        if n in vis:
            continue
        vis.add(n)

        self.steps.append((path, False))

        if n == goal:
            self.best_path = path
            self.steps.append((path, True))
            return

        for nxt, _ in graph.get(n, []):
            if nxt not in vis:
                q.append((nxt, path + [nxt]))
```

Implementation of DFS

```python
# ============================================================
# Depth-First Search (DFS)
# ============================================================
def dfs(self, start, goal):
    stack = [(start, [start])]
    vis = set()
    self.steps = []

    graph = self.build_graph()

    while stack:
        n, path = stack.pop()

        if n in vis:
            continue
        vis.add(n)

        self.steps.append((path, False))

        if n == goal:
            self.best_path = path
            self.steps.append((path, True))
            return

        for nxt, _ in graph.get(n, []):
            if nxt not in vis:
                stack.append((nxt, path + [nxt]))
```

Implementation of UCS

```python
# ============================================================
# Uniform Cost Search (UCS) - Visual Version
# ============================================================
def ucs(self, start, goal):
    self.is_ucs_running = True
    pq = [(0, start, [])] # cost = time so far
    vis = {}
    self.steps = []

    while pq:
        cost, node, path = heapq.heappop(pq)
        if node in vis and vis[node] <= cost:
            continue
        vis[node] = cost
        cp = path + [node]
        self.steps.append((cp, False))
        if node == goal:
            self.best_path = cp
            self.steps.append((cp, True))
            break
        for nxt, e in self.build_graph().get(node, []):
            length = e["length"]
            traffic = e.get("traffic", 0)

            edge_cost = length * (1 + traffic / 10)

            heapq.heappush(pq, (cost + edge_cost, nxt, cp))
```

Implementation of A*

```python
# ============================================================
# A* Search Algorithm
# ============================================================
def astar(self, start, goal):
    def h(n):
        return (self.nodes[n]["pos"] - self.nodes[goal]["pos"]).manhattanLength()

    pq = [(h(start), 0, start, [start])]
    seen = {}
    self.steps = []

    graph = self.build_graph()

    while pq:
        _, cost, n, path = heapq.heappop(pq)

        if n in seen and seen[n] <= cost:
            continue
        seen[n] = cost

        self.steps.append((path, False))

        if n == goal:
            self.best_path = path
            self.steps.append((path, True))
            return

        for nxt, e in graph.get(n, []):
            length = e["length"]
            traffic = e.get("traffic", 0)

            edge_cost = length * (1 + traffic/10)   # real cost

            g2 = cost + edge_cost
            f2 = g2 + h(nxt)

            heapq.heappush(pq, (f2, g2, nxt, path + [nxt]))
```

Directed and Undirected Graph Implementation

Directed and Undirected Graph Implementation

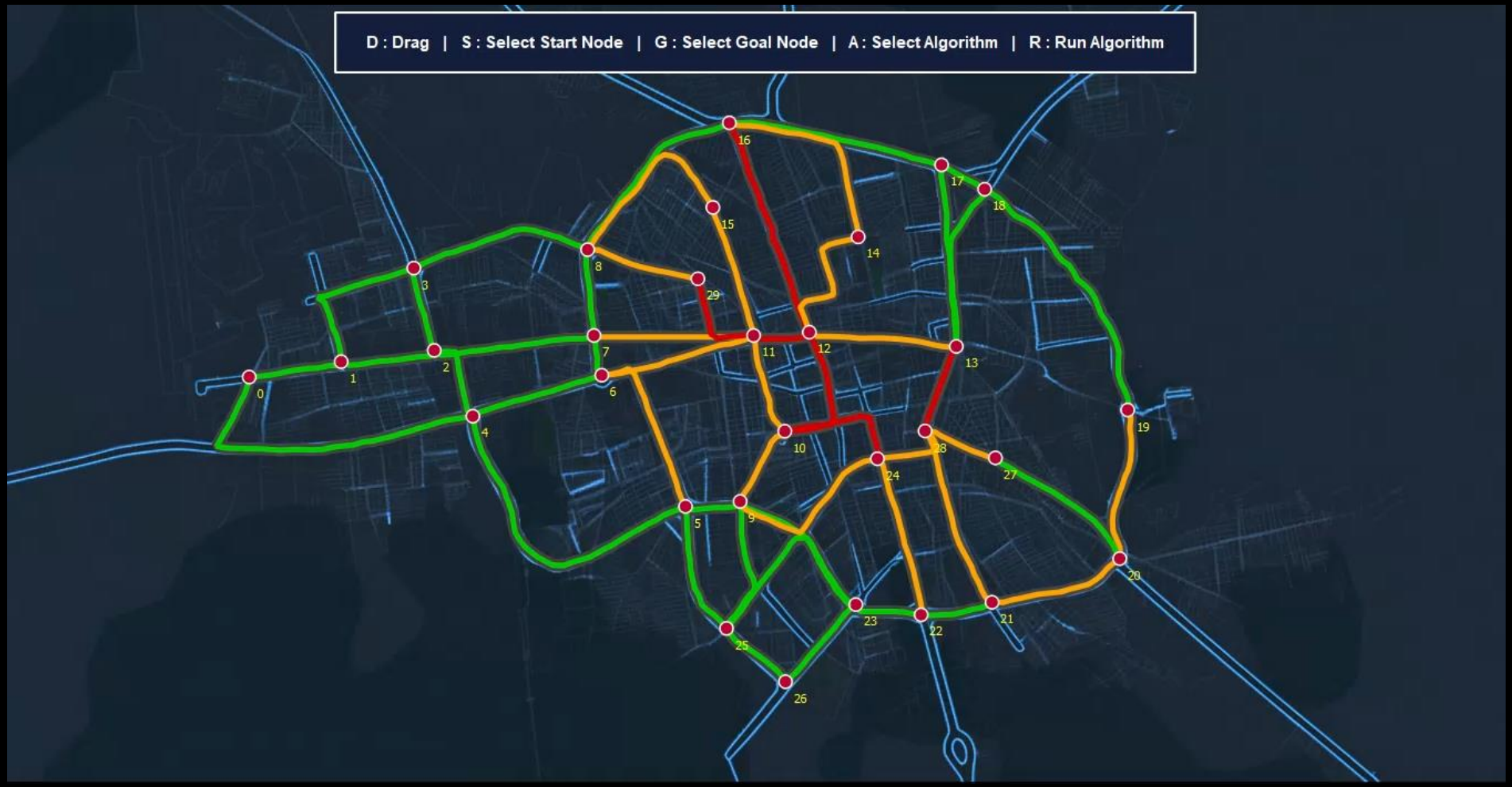
The constructed network can be represented as both directed and undirected graphs.

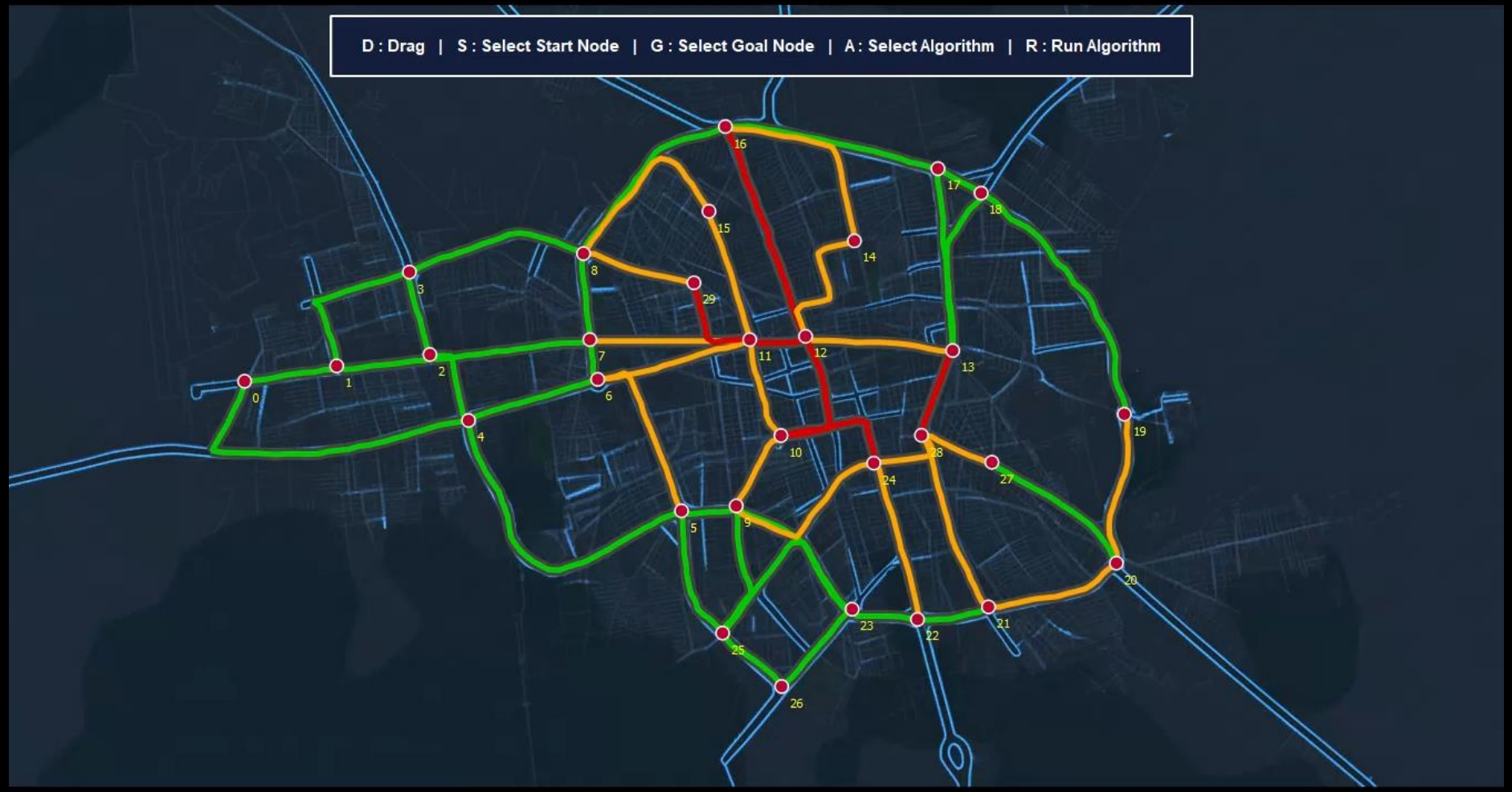
In the directed graph, each edge has a defined direction from a source node to a destination node.

In the undirected graph, edges allow movement in both directions between nodes.

Undirected Graph Implementation (For BFS)

Note: This algorithm ignores distance, traffic, and cost when selecting a path. The chosen path is the first one that reaches the goal with the minimum number of edges.



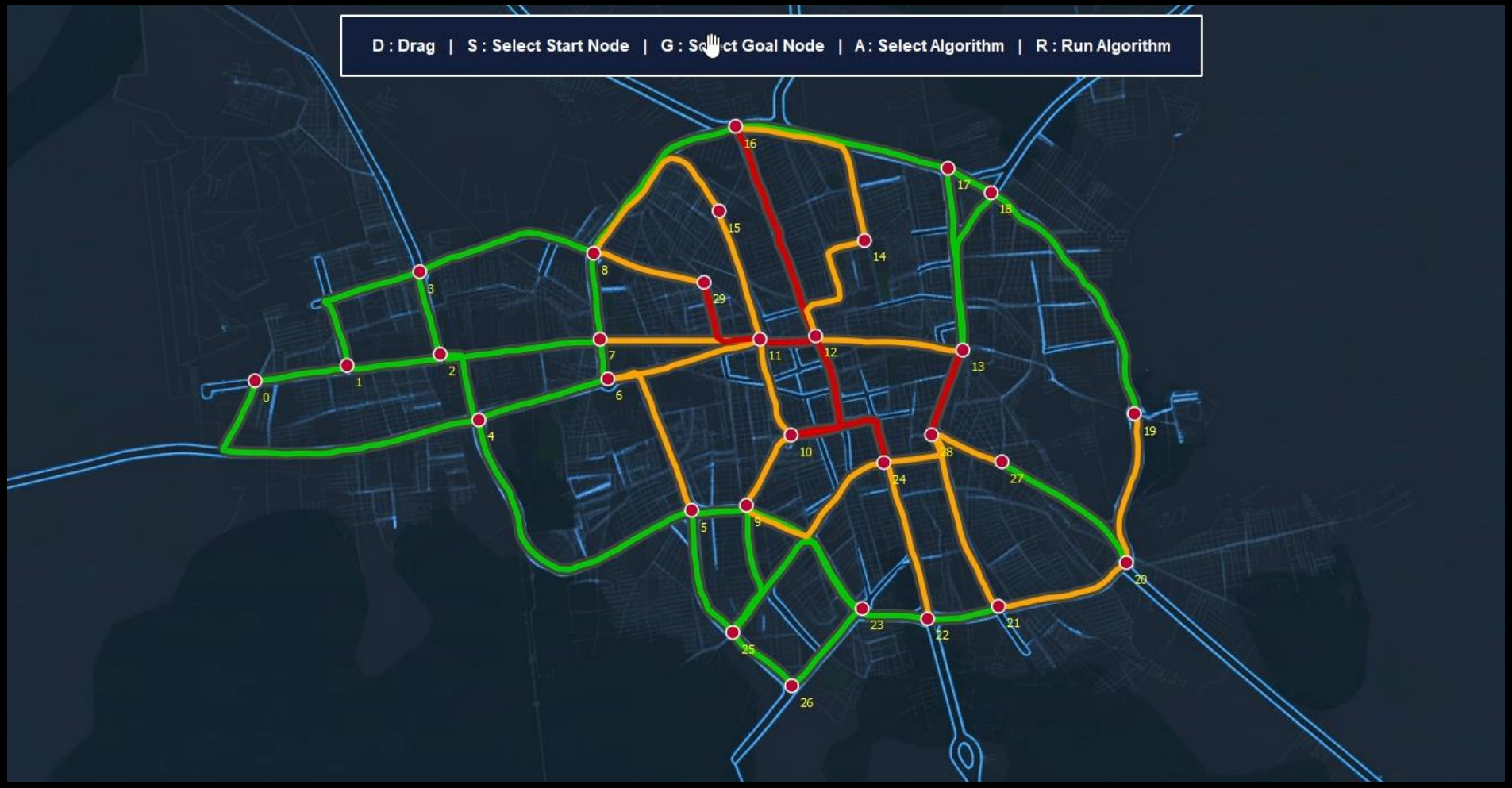


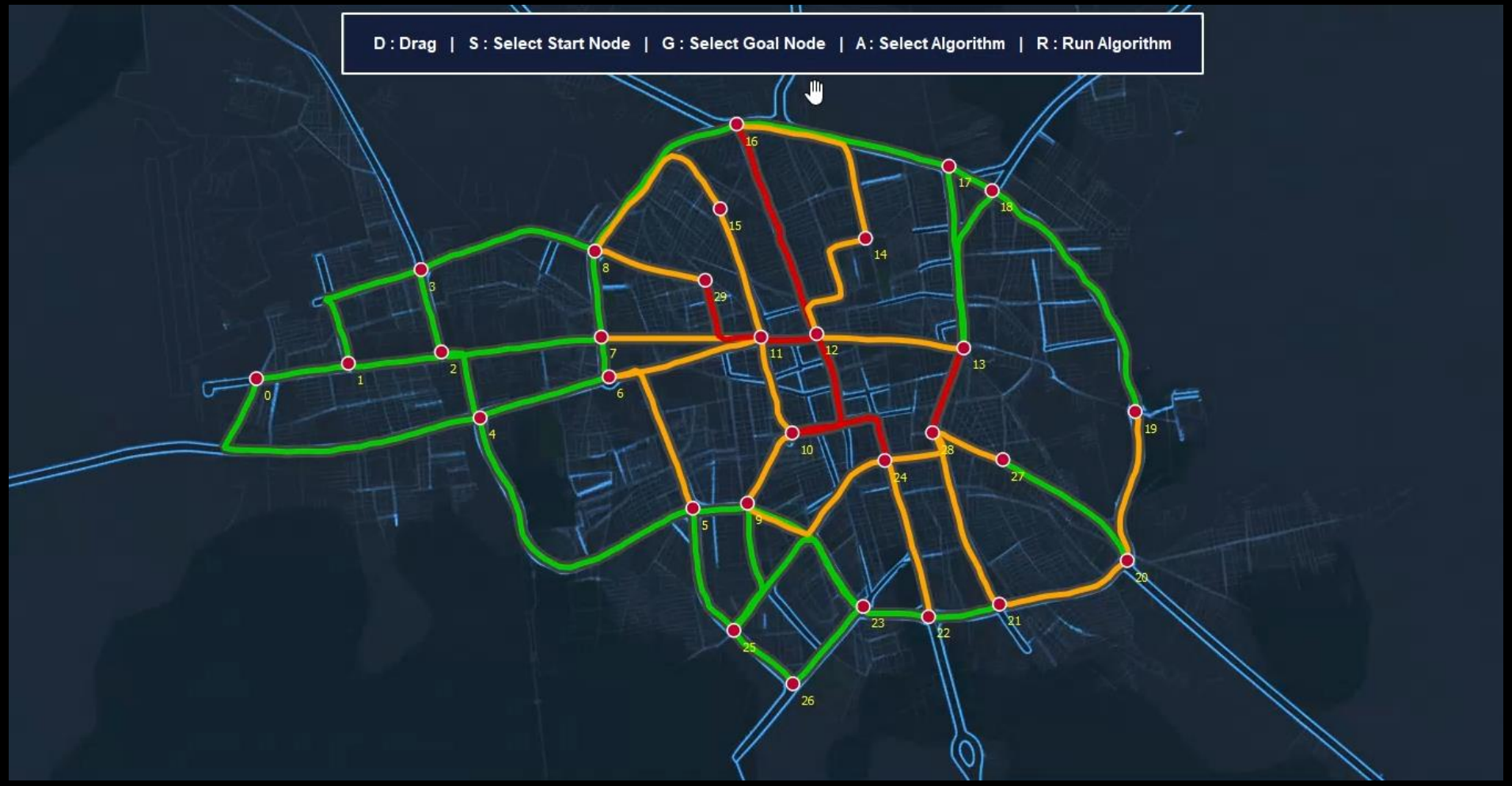
Directed Graph Implementation (For BFS)

Note: This algorithm ignores distance, traffic, and cost when selecting a path. The chosen path is the first one that reaches the goal with the minimum number of edges.

Undirected Graph Implementation (For DFS)

Note: In this implementation, DFS explores the graph by going as deep as possible along each branch before backtracking.
The algorithm does not guarantee an optimal path in terms of distance, traffic, or cost.



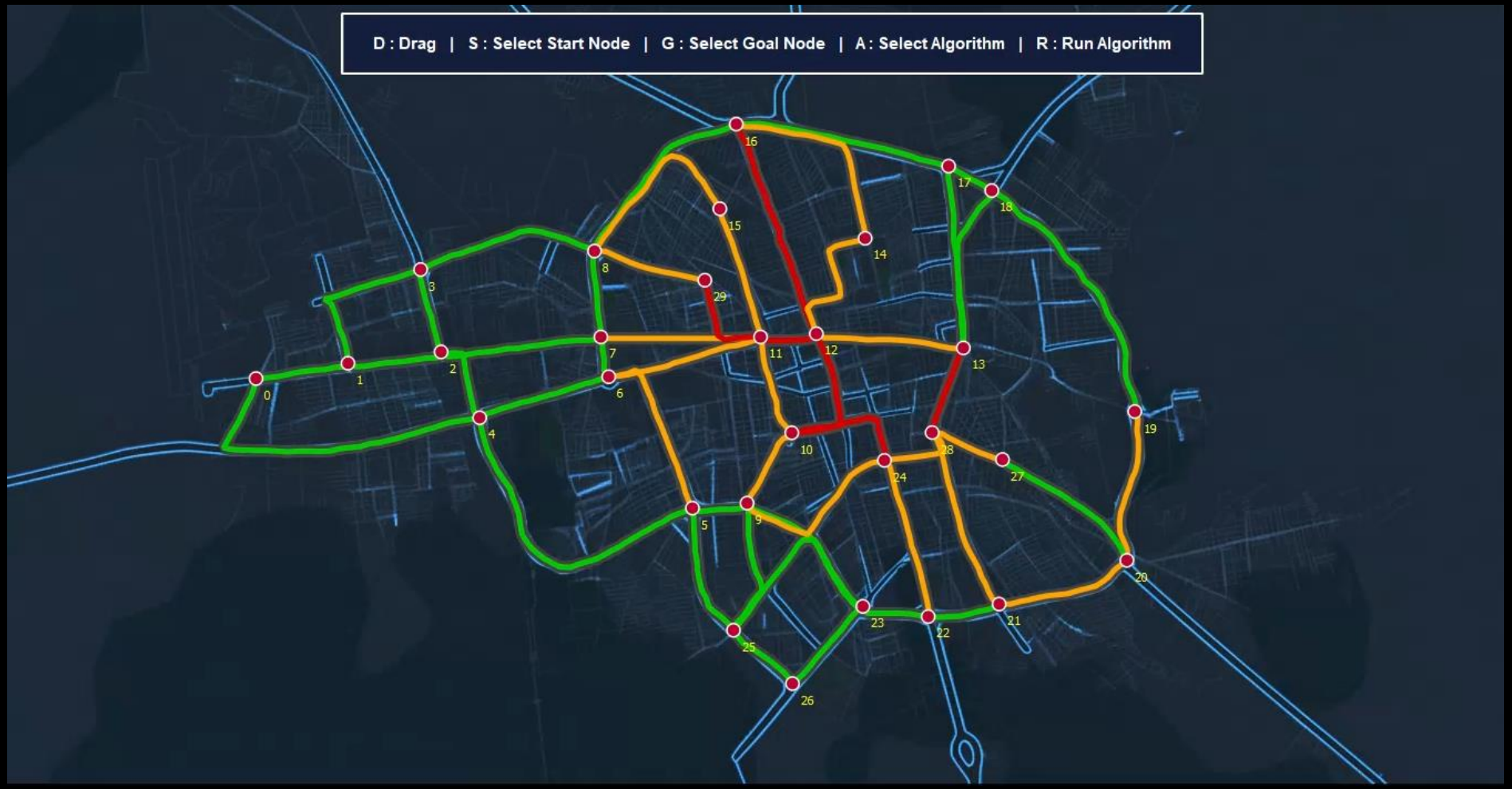


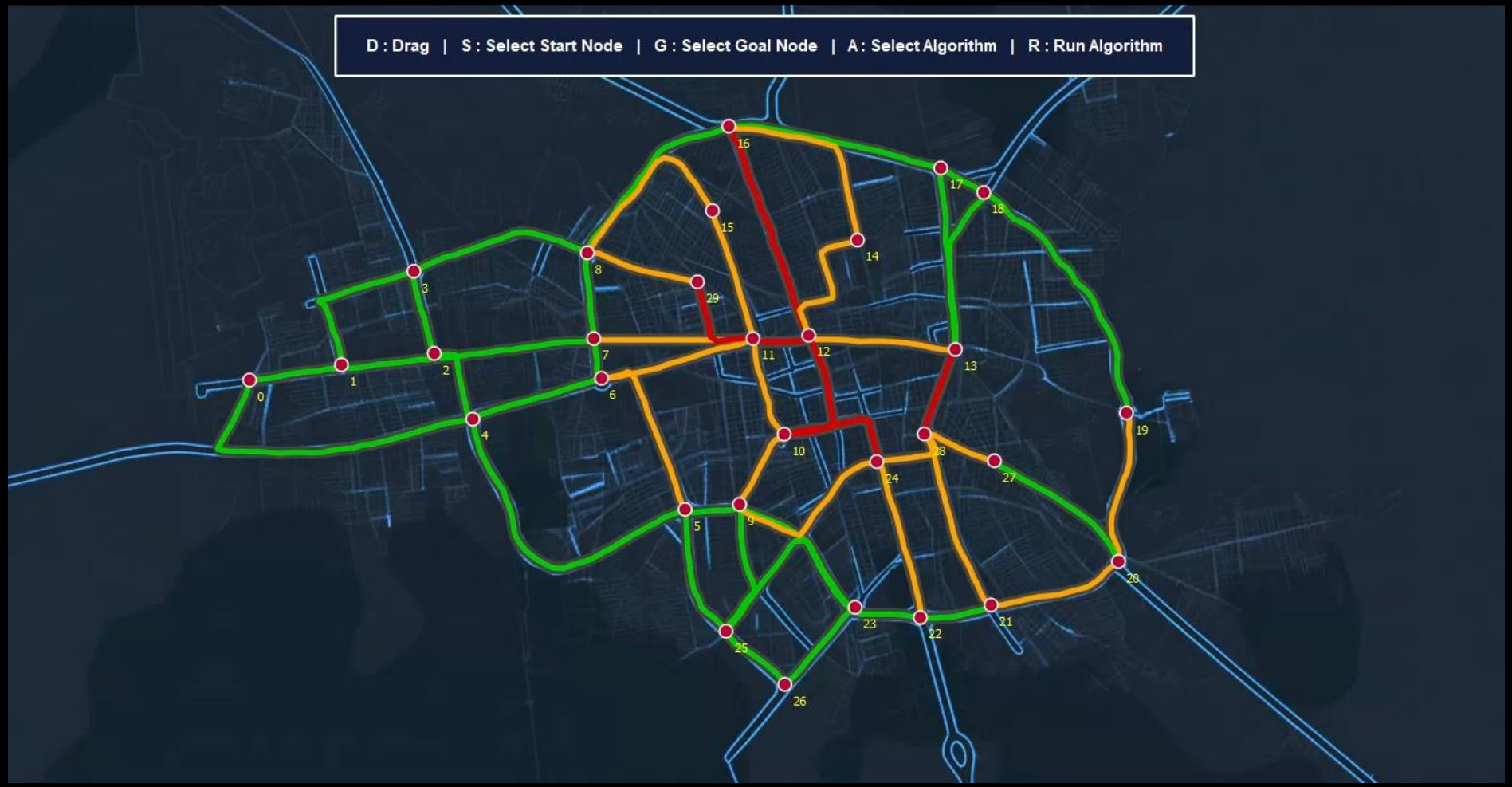
Directed Graph Implementation (For DFS)

Note: In this implementation, DFS explores the graph by going as deep as possible along each branch before backtracking.
The algorithm does not guarantee an optimal path in terms of distance, traffic, or cost.

Undirected Graph Implementation (For UCS)

NOTE: For UCS, the graph is implemented as an undirected graph where all edges are bidirectional.
The algorithm selects paths based on cumulative cost, which may include distance and traffic.



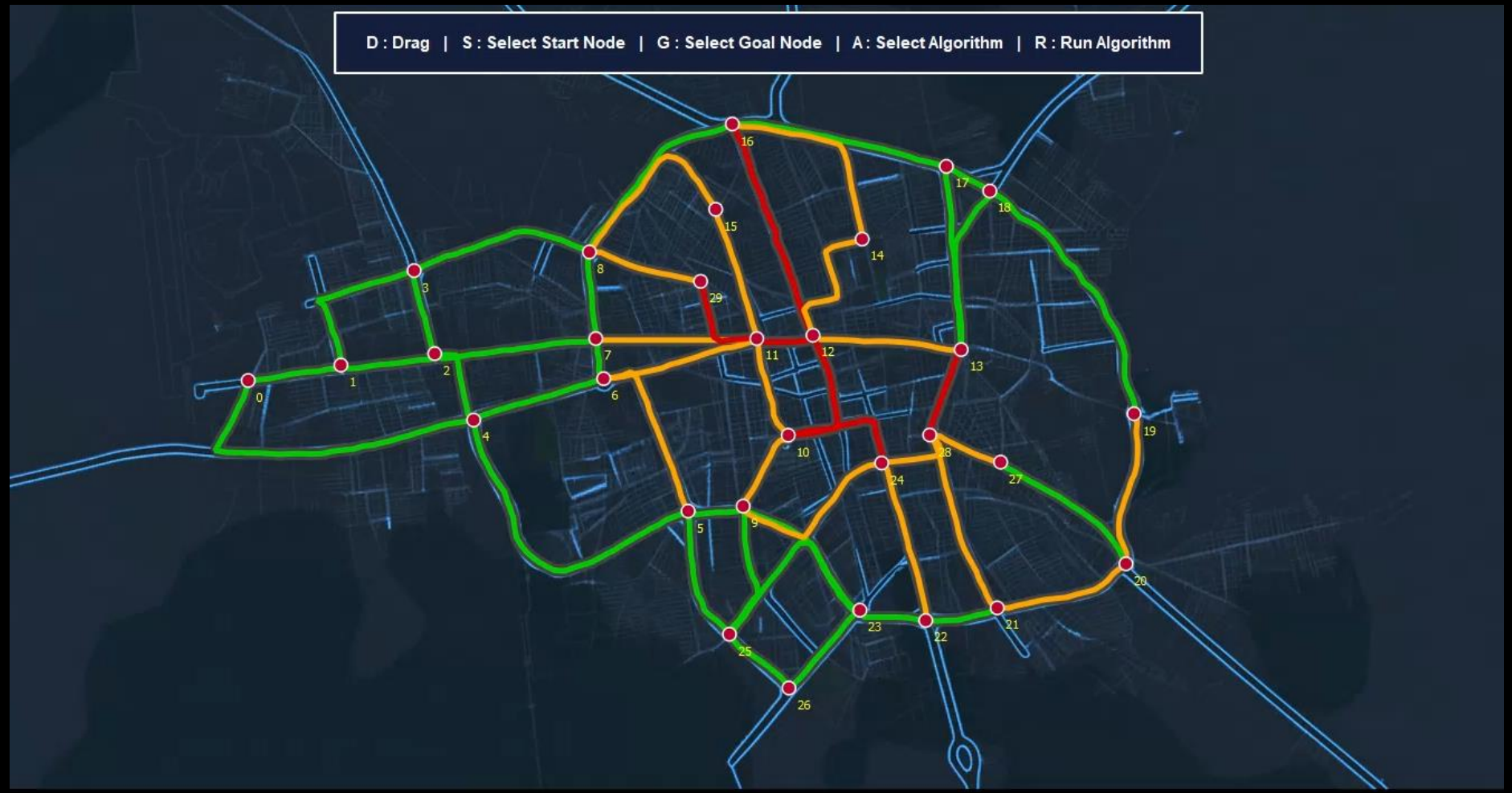


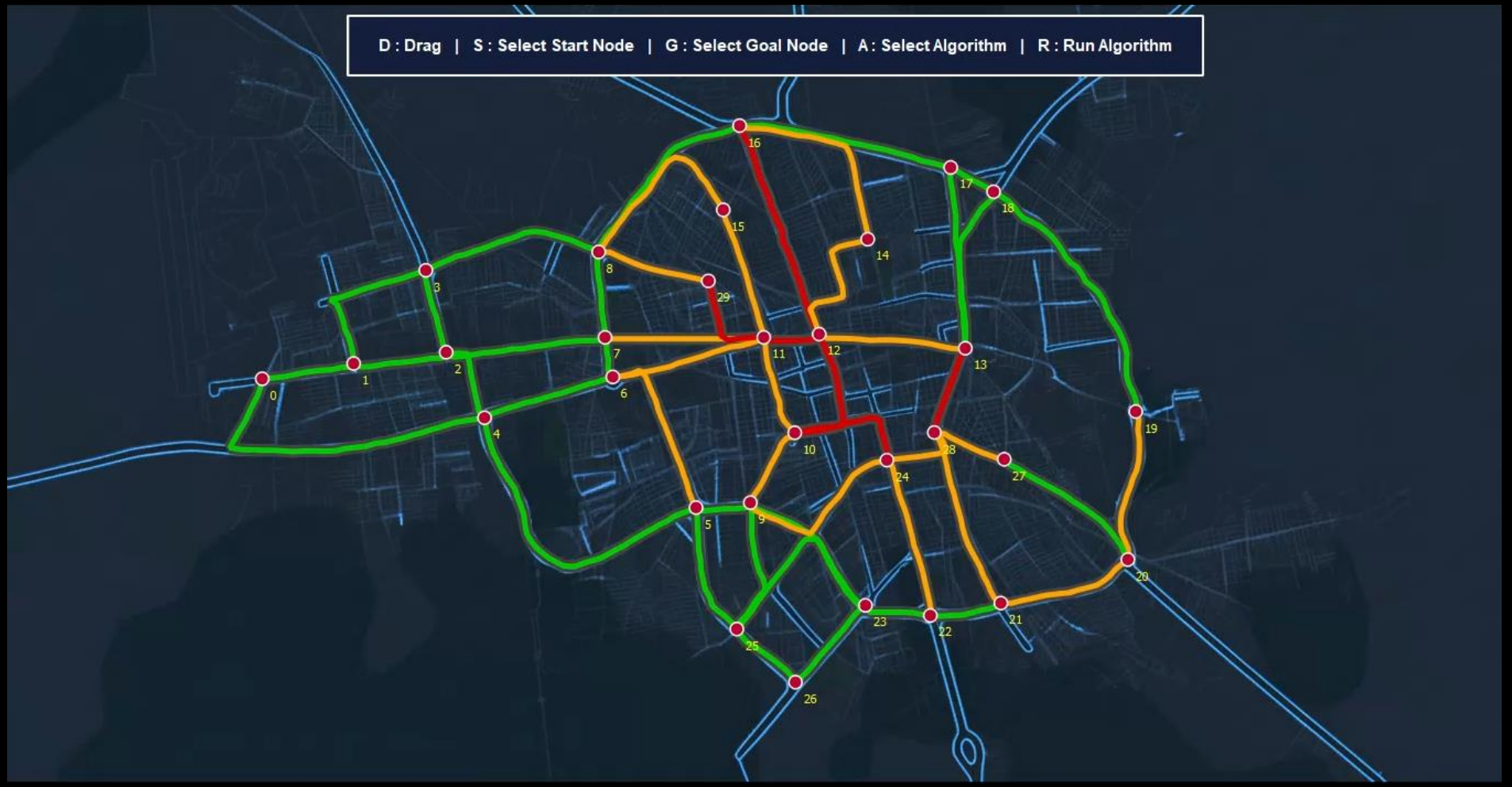
Directed Graph Implementation (For UCS)

NOTE: For UCS, the graph is implemented as an undirected graph where all edges are bidirectional.
The algorithm selects paths based on cumulative cost, which may include distance and traffic.

Undirected Graph Implementation (For A*)

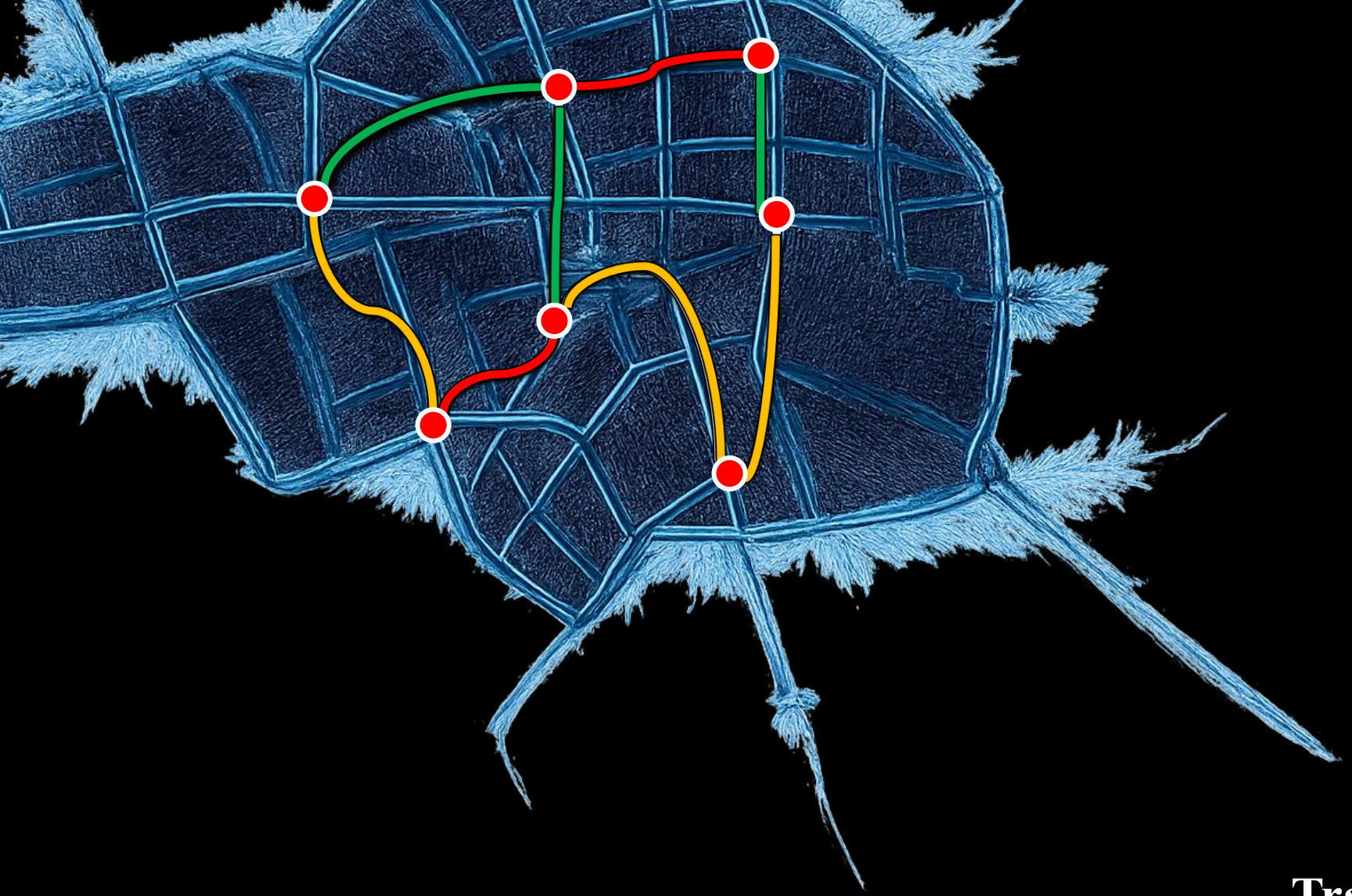
NOTE: For A*, the graph is implemented as an undirected graph where all edges are bidirectional.
The algorithm combines path cost and a heuristic function to efficiently guide the search.





Undirected Graph Implementation (For A*)

NOTE: For A*, the graph is implemented as an undirected graph where all edges are bidirectional.
The algorithm combines path cost and a heuristic function to efficiently guide the search.



Traffic Cost Calculation

Traffic Cost Calculation

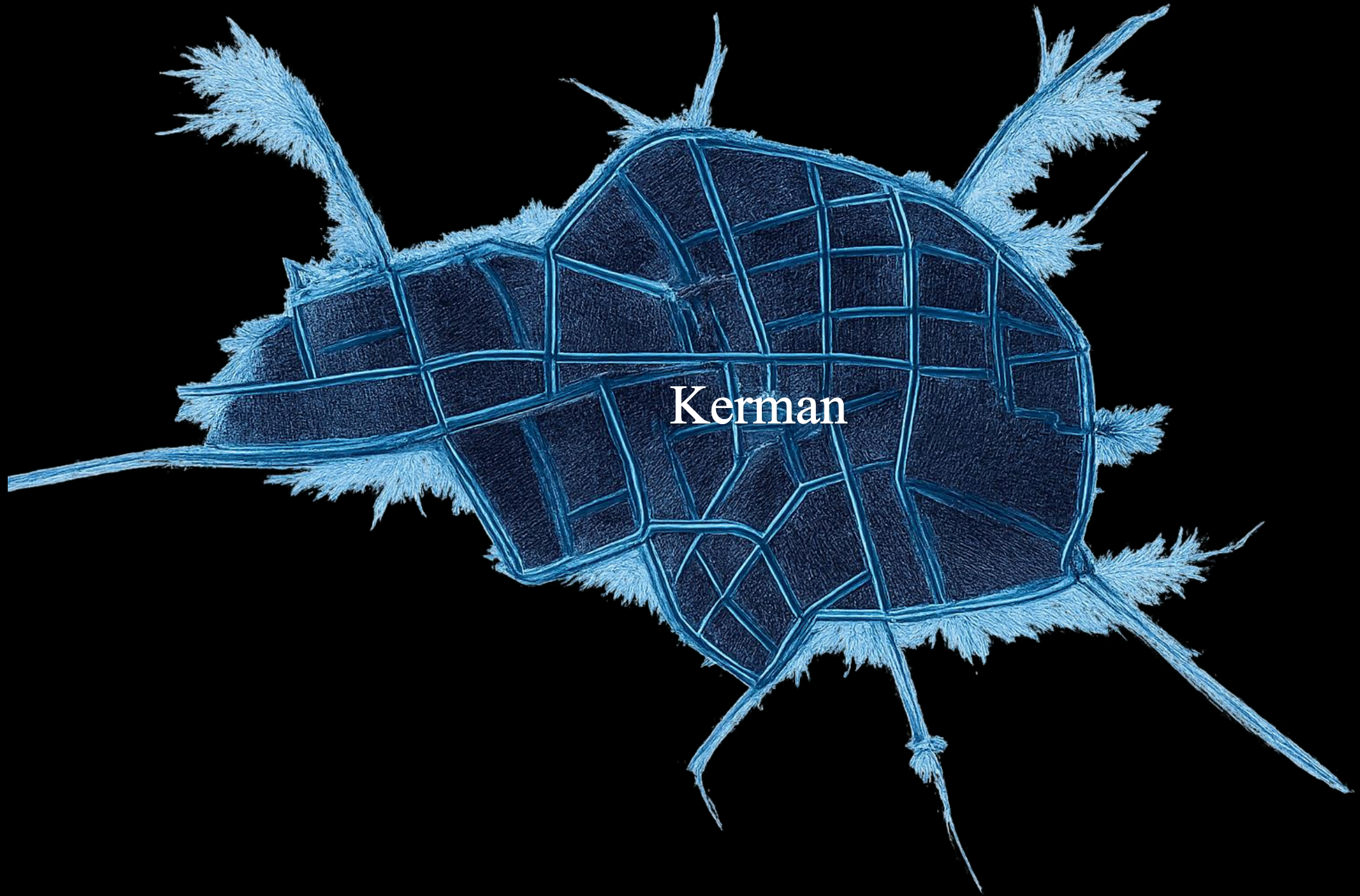
In this project, traffic is used as a factor that increases the cost of traversing each edge.

The final edge cost is calculated using the formula:

Cost = Distance × (1 + Traffic / 10)

Higher traffic results in higher traversal cost, encouraging the algorithms to avoid congested paths.

This cost model is used only in UCS and A* algorithms, while BFS and DFS ignore traffic and cost values.


Kerman